

Prep work for podcast

“how can SpecKit be applied to legacy application”

Authors: Amit with Wai team

Planning for the conversation

- Topic: how can SpecKit be applied to legacy application
- Demonstrate one application with talking points
- white-labelled experiment = `git@github.com:amitrajpurkar/sprboot_api_cktBrkr.git`
- Sequence
 - Context/ background ... pages 3-6 : xxx minutes... 20%
 - Where and how this experiment fits ... pages 7 : xxx minutes... 10%
 - Journey of this experiment ... pages 8-17 : xxx minutes... 60%
 - Summary and other observations ... pages 18 : xxx minutes... 5%
 - Ending thoughts ... pages 19-21 : xxx minutes... 5%

Context for Legacy Application

A **legacy application** is a software system that:

- Is **critical to business operations**, but
- Is **difficult, risky, or costly to modify, maintain, or replace**, due to technical, organizational, or strategic limitations.

In practice, something becomes “legacy” when **change velocity slows down** while **business dependency remains high**.

- ❌ “Old software = legacy”
→ Not necessarily. Well-maintained older systems may not be legacy.
- ❌ “Legacy means useless”
→ Many legacy systems are **mission-critical** and extremely valuable.
- ❌ “We must rewrite everything”
→ Full rewrites are often the **riskiest option**.

Ask these questions:

- Is the system **still delivering business value**?
- How often does it need to change?
- What is the **risk of failure** vs. risk of change?
- Is the main pain **technology, architecture, people, or business fit**?
- What is the **time horizon** (1 year vs. 5 years)?

The answers usually determine the strategy.

• Qualities of Legacy application

- Technical Characteristics: outdated tech, tight coupling, poor architecture/design, low testability
- Human factors: loss of institutional knowledge; skill scarcity
- Operational Characteristics: high risk of change, costly maintenance
- Business & Strategic factors: misaligned with current business needs, compliance/ security risks

• Different ways to address Legacy application

- Leave as is
- Refactor/ Modernize
- Re-platform
- Rewrite/ Replace
- Strangler pattern

• Observation:

- Investigate both code as well as business requirements; get latest business requirements; use AI to analyze -> document -> get specs; AI for wrap-around testing; tackle one feature at a time;

Estimate talk time: 3-5 minutes ?

Context for AI Code Assistants

- AI does not replace a task; it augments and helps with automation; work is still done by the developer
- Developer is responsible for the code generated by the AI Assistant based on prompts provided
- Large number of Assistants are available; all are fairly similar; its a developers choice to pick one as per their style and need
- These code assistants can provide in-line suggestions while the developer is writing code or they allow developers to chat with it allowing AI to write code.
- SpecKit works with the chatbot option to generate the specifications; The developer along the way will need to use both styles of interaction with AI to reach the final outcome

Estimate talk time: 3-5 minutes ?

System prompt vs User prompt

A **system prompt** is an instruction that **defines the AI's role, behavior, and constraints**. It sets the *rules of engagement* for the assistant.

A **user prompt** is the **specific request or question** the developer asks the AI.

Aspect	System Prompt	User Prompt
Scope	Global / session-wide	Local / per request
Who sets it	Tool, platform, developer	End user / developer
Priority	Highest	Lower
Purpose	Define behavior & rules	Request a task
Persistence	Long-lived	Short-lived
Visibility	Often hidden	Always visible

Understanding the distinction helps you:

- Design **better AI-assisted development workflows**
- Use system prompts to enforce **coding standards and safety**
- Keep user prompts focused on **intent, not behavior**
- Achieve more **consistent and predictable outputs**

- System prompts establish how the AI Assistant should behave; provides global context, sets priorities; remains consistent across many user prompts
- User prompts describe what task to perform right now, the context, inputs, constraints for that task
- In a way, SpecKit helps provide system prompts in a standardized way; streamlines development practice across a team of developers.

Estimate talk time: 3-5 minutes ?

Context for SpecKit

Spec Kit is built around the idea of **Spec-Driven Development (SDD)** — a new AI-centric development workflow that **makes specifications the true source of truth** and uses them to **drive the generation of actual working code** rather than just guide it.

-  **Core Ideas**
- **“What before How”** – Emphasize *what* the system should do (the intent) before *how* it should do it.
- **Executable Specs** – Specifications aren’t just documentation; they are refined and used to *generate implementation* through AI agents.
- **Structured Workflow with AI Agents** – There’s a multi-stage workflow:
 - **Constitution** – Set project principles
 - **Specify** – Write the product spec
 - **Plan** – Add tech context (stack, architectural decisions)
 - **Tasks** – Break work into actionable steps
 - **Implement** – Have AI assistants build the code from tasks
- **AI-first orchestration** – Heavy reliance on AI for interpreting specs, planning, and executing tasks, with humans in a review/verification loop.
-  **Why This Matters**

Traditional dev is often *code-centric* — start coding and defer specifications/documentation. SDD flips that: *specs drive the entire creation process*, and code is synthesized to match the spec.

- SpecKit provides set of templates for AI Code Assistants (pull the latest).. use CLI to configure these templates for your Code Assistant or setup manually
- Philosophy: document the specification, plan, tasks very well before making code changes
- What this means for a developer: **“document accurately before you code”**

Dimension	Spec-Driven with AI	BDD with Human Developers
Primary Artifact	Rich structured specifications	Behavior scenarios (Gherkin tests)
Driver of Work	AI agents generating code	Humans writing and implementing tests
Feedback	AI-built code validated by humans	Tests failing/passing validate code
Alignment	Specification → implementation	Conversation → scenario → implementation
Automation Scope	High (from spec to code)	High for tests, lower for full code
Risk	Misinterpretation by AI	Miscommunication among humans

Estimate talk time: 3-5 minutes ?

About the Experiment

Migrating a sample application from old tech stack to new

- Why is this app legacy? a working application written in 2020 on older tech stack; newer versions of Java, Gradle do not support that stack; one of the key libraries is no longer supported today; a demo app with rest APIs which are expected to run 24/7 with a high transaction volume of up to 3,000 transactions per minute.
- This experiment was started before I knew about SpecKit; hence it used AI Assistant following a plan and set sequence of instructions
- Refactoring and modernization technique was used
- Key points:
 - Initial app was developed by myself in 2017; so the entire codebase as well as business requirements were well known; this was not a black-box
 - Target tech stack after upgrade was also known before hand
 - Experiment was with sequencing instructions to the AI Assistant and testing each step of the way

Planning to work with AI

- How I planned/ broke-down migration effort into phases and steps for making changes
- Sequencing and prioritization of the phases and steps
- Making the AI analyze the current codebase itself and match its analysis with my own experience creating the original app
- Getting the ground ready to let AI start making changes
- Key point —> we don't start coding directly when using AI.. we need to plan, document and be very specific on what we need to do.

Glimpse of First Analysis

AI reviews, shows findings, Developer verifies

Pattern	Implementation	Quality
Circuit Breaker	Hystrix + Failsafe	★★★★★ Excellent dual implementation
Aspect-Oriented Programming	Spring AOP for logging	★★★★★ Well-structured aspects
Builder Pattern	Product model	★★★★★ Clean implementation
Configuration Properties	Type-safe config binding	★★★★★ Exemplary use of <code>@ConfigurationProperties</code>
Repository Pattern	Spring Data MongoDB	★★★★★ Standard implementation
Dependency Injection	Constructor/field injection	★★★ Mixed approach (could be improved)

Package Organization:		
com.anr/		
├─ Bootstrap.java		# Main application entry point
├─ common/		# Utilities (SBUtl)
├─ config/		# Configuration classes (9 files)
├─ controller/		# REST controllers (4 files)
├─ exception/		# Exception handling
├─ localmdb/		# MongoDB domain & repositories
├─ logging/		# AOP logging aspects (5 files)
├─ model/		# Response models
└─ service/		# Business logic layer (3 files)

Technology	Version	Status	Assessment
Java	14.0.2	⚠ Outdated	LTS versions: 11, 17, 21 recommended
Spring Boot	2.3.3.RELEASE	⚠ Outdated	Current: 3.x (requires Java 17+)
Gradle	6.6	⚠ Outdated	Current: 8.x
Hystrix	2.2.4.RELEASE	🛑 Deprecated	Maintenance mode since 2018
Failsafe	1.1.0	⚠ Very old	Current: 3.x
MongoDB Driver	Spring Data	✅ Good	Properly abstracted
SpringDoc OpenAPI	1.4.4	⚠ Outdated	Current: 2.x
JaCoCo	0.8.5	⚠ Outdated	Current: 0.8.11+
ArchUnit	0.9.1	🛑 Very old	Current: 1.x
Lombok	Included	✅ Good	Reduces boilerplate
Gson	2.8.6	⚠ Outdated	Consider Jackson (already included)
Guava	29.0-jre	⚠ Outdated	Current: 32.x

- Ask AI Code Assistant to analyze the codebase
- As a developer, verify the findings documented by AI
- In my case, being the original developer of this application written in 2020, I was able to verify accuracy of the analysis

First Analysis continued

9.1 What's Good (Still Relevant in 2025)		
Pattern/Practice	Status	Notes
Circuit Breaker Pattern	✓ Timeless	Core resilience pattern
AOP for Cross-Cutting Concerns	✓ Current	Still best practice
Type-Safe Configuration	✓ Current	Spring Boot standard
Repository Pattern	✓ Current	Clean architecture
Structured Logging	✓ Current	Observability requirement
API Documentation	✓ Current	OpenAPI standard

9.2 What's Outdated		
Aspect	Issue	Modern Alternative
Hystrix	Deprecated	Resilience4j
Java 14	Non-LTS	Java 17 or 21 LTS
Spring Boot 2.3	Old	Spring Boot 3.x
JUnit 4	Legacy	JUnit 5 (Jupiter)
Mockito 1.x	Ancient	Mockito 5.x
WAR Packaging	Traditional	JAR + Container
Servlet Stack	Blocking I/O	Reactive (WebFlux) optional

- Even before I showed AI my plan for migration, it had listed correctly which features were good and which outdated.
- This also helped me update my own plan for migration

After migration was completed

Backend		
Component	Technology	Version
Framework	Spring Boot	3.2.10
Language	Java	21
Build Tool	Gradle	9.1.0
Database	H2	2.2.224
ORM	JPA/Hibernate	-
API Docs	OpenAPI/Swagger	2.3.0
Circuit Breaker	Resilience4j	2.2.0

Frontend		
Component	Technology	Version
Framework	React	18.2.0
HTTP Client	Axios	1.6.0
Build Tool	React Scripts	5.0.1
Styling	CSS3	-
State Management	React Hooks	-

Drafting the plan

- Step 1: ask AI to analyze codebase; describe what this application is and what key features it has (NOTE: this later part is because I was the original developer)
- Step 2: ask AI to create a step by step plan for migrating this application to latest stack and document this plan in a markdown file. Give it a hint to use gradle 9, java 21
- Step 3: ask AI to create separate migration plan for circuit breaker migration and save that in another markdown; give hint to use resilience4j (Here as well I had researched this lib before hand)
- Step 4: ask AI to create another plan for changing backend from mongodb to H2
- NOTE: separate research was done beforehand on steps 2, 3, 4; experiment was observing how AI was addressing these tasks

Journey of changes

Comparing my approach with speckit

- Further experimenting on other greenfield projects using speckit; the sequential plan I used here was in line with speckit's specify, clarify, plan commands
- The features or changes needed were called out in steps 2, 3, 4
- Also fair amount of interaction went behind each steps to get it to a desired level: java/springboot-upgrade; circuit-breaker-upgrade; database-upgrade;
- One important distinction; in this experiment I was having AI implement each step fully at a time; unlike executing "task-command" followed by "implement";
- Also for each code change AI was making, I was verifying those changes, testing with unit tests and then committing changes that looked good.

Backend working with new stack

Compare with speckit

- In the migration experiment I had fair amount of interaction with AI to reach on a good enough upgrade; later experiments using speckit proved that if specification and plan is build with careful details, these iterations can be controlled
- It took me 3-4 weeks to complete steps 2, 3, 4 as outlined before. — speckit can help reduce this time
- Verifying changes made by AI using test cases was also important; on some occasions I also had AI generate more test cases. I was manually checking these in the Windsurf to see how these tests ran.



Increase testability

- My initial test cases in the original application were only checking on the service layer, repository layer, and Aspects
- With code changes made by AI I started adding more tests around utilities, service layer and even controller
- Next I used AI to increase my test coverage to a higher level than before
- Classifying test cases per feature using BDD style test cases is helpful for pair programming with AI; more so as you are detailing fine-grain specifications; so its better to test is the code is satisfying those specifications.

Adding new feature

- This app was not having a client for the API — either a web-frontend or a CLI; so next came feature additions now that the essential upgrade was done.
- I asked AI to add a frontend UI in ReactJS and that it should be a single page application
- This also ran through numerous iterations till desirable frontend UI was working

Adding documentation for the app

- Over the 5 week period working with AI to migrate the legacy application to new tech stack, I had been making my notes each day. After both backend and frontend were working as desired, now was a good time to capture all notes in the repository itself
- I had AI generate documentation, of my journey — project summary, architecture, migration guides for gradle, resilience4j, H2; implementation guides, test summaries, code review, quick start guides, etc

Re-arrange docs to Mkdocs

- For my experiment, I hand crafted the mkdocs for this application using documentation generated by AI
- I arranged the pages, the navigation, wrote my own index page and few others to organize documentation in certain way
- With thoughtful planning this could be achieved by asking AI generate documents in this layout and structure.

Summarizing this experiment

- As my first experiment playing with AI Code Assistant, this did give me insight on how I should be interacting with AI
- I experimented with SpecKit separately on simple example projects from scratch
- If I need to experiment on a large legacy system and also make use of SpecKit; my personal approach will be to isolate one feature or aspect of that legacy app and work with AI using SpecKit to understand and document the behavior of that feature; verify with business partners before making any changes.

Old advice is still relevant

“Working effectively with Legacy Code” by Michael Feathers (2004)

- First add tests and then do your changes
- Identify places to alter program behavior without changing code (“seams”)
- Create tests that characterizes behavior of the piece of code
- Use sprout and wrap techniques
- Sprout: create code in separate module/ package; test it; identify insertion point; call it from legacy code
- Wrap: rename old method you want to wrap; create a new method with same signature as old; call old method from new; put logic before & after call; like AOP

From other developers

Provide references to what other developers are researching

- Follow podcasts/ spotify/ YouTube from leading developer organizations: Thoughtworks, MartinFowler, Info-Q, ChangeLog, Software Engineering podcast; — plus the key sources: Anthropic, OpenAI, Google, Microsoft
- <https://podcasts.apple.com/us/podcast/how-ai-will-change-software-engineering-with-martin-fowler/id1769051199?i=1000737446931>
- <https://martinfowler.com/articles/legacy-modernization-gen-ai.html>
- <https://www.thoughtworks.com/en-in/insights/podcasts/technology-podcasts/using-generative-ai-legacy-modernization>
- https://youtu.be/6aD4c_izpnE?si=visMMXLY5mkJrQ8j
- https://youtu.be/yNH52oDpIKw?si=TFH8X0ErUW6E7-_O
- <https://www.deeplearning.ai/>. <https://www.anthropic.com/learn>

Ending thoughts

- AI is still evolving at a very rapid pace.. standardization in coding practice using AI is in its infancy
- Developers need to try the tools and experiment with different approaches as they surface.
- SpecKit is providing one such path; Claude Code is also using a very similar approach using “skills, scripts, templates, workflows” (comparing to memory, scripts, templates, workflows in SpecKit)
- If you don’t use SpecKit like approach, the developer needs to babysit with AI for a long time